VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JNANA SANGAMA" Belagavi – 590018, Karnataka



A

MAJOR PROJECT REPORT

on

"VEERDRISHTI: COMBAT VISION SYSTEM"

Submitted in partial fulfillment for the award of the degree in

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by

GAURAV CHAUHAN	1BH23CS024
GAURAV SINGH	1BH23CS025

Under the Guidance of

Ms. Sapna N

Asst. Prof., Dept. of CSE

Department of Computer Science and Engineering

BANGALORE TECHNOLOGICAL INSTITUTE

(NAAC Accredited, An ISO 9001:2015 Certified Institute)

Kodathi Village, Varthoor Hobli, Bengaluru East Taluk, Bengaluru Urban District

Bengaluru, Karnataka – 560035.

2026–2027

BANGALORE TECHNOLOGICAL INSTITUTE
www.btibangalore.org | principal@btibangalore.org | 7090404050

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the Major Project Report entitled **"VeerDrishti: Combat Vision System"** carried out by **Gaurav Chauhan [1BH23CS024]** and **Gaurav Singh [1BH23CS025]**, the bonafide student of **Bangalore Technological Institute, Bengaluru** in partial fulfilment for the award of **Bachelor of Engineering** in **Computer Science and Engineering** of the **Visvesvaraya Technological University, Belagavi** during the academic year **2026–2027**. Thus, it is certified that all corrections/suggestions indicated for the Internal Assessment have been incorporated in the report. The Major Project Report has been approved as it satisfies the academic requirements in respect of major project prescribed for the said degree.

Ms. Sapna N	**Mrs. Swaroopa K**
Asst. Prof., Project Guide	Asst. Prof., Project Coordinator
Dept. of CSE	Dept. of CSE
Dr. D. Jerline Sheebha Anni	**Dr. H. S. Nanda**
HoD, Dept. of CSE	Principal

VIVA

Name of Examiners	**Signature with date**
1. _____	_____
2. _____	_____

BANGALORE TECHNOLOGICAL INSTITUTE
BANGALORE TECHNOLOGICAL INSTITUTE
www.btibangalore.org | principal@btibangalore.org | 7090404050

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

We, the students of final year Computer Science and Engineering, **Bangalore Technological Institute, Bengaluru**, hereby declare that, the Major Project Report work **"VeerDrishti: Combat Vision System"** embodies the report of our research work carried out under the guidance of **Ms. Sapna N**, Asst. Prof., **Department of Computer Science and Engineering, Bangalore Technological Institute, Bengaluru** as partial fulfilment of requirements for the **Bachelor of Engineering Degree in Computer Science and Engineering of Visvesvaraya Technological University, Belagavi**, during the academic year **2026–2027**.

We also declare that to the best of our knowledge and believe the work reported here does not form part of any other project on the basis of which a degree or award was conferred on an early occasion of this by any other.

Date: _____

Place: Bengaluru

Student Name	USN	Signature
GAURAV CHAUHAN	1BH23CS024	.....
GAURAV SINGH	1BH23CS025	.....

ACKNOWLEDGEMENT

Any achievement, be it scholastic or otherwise does not depend solely on the individual efforts but on the guidance, encouragement and cooperation of intellectuals and elders. We would like to take this opportunity to thank them all.

We heartily extend my words of gratitude to the Assistant Professor **Ms. Sapna N**, Major Project Guide for his valuable advice, encouragement and suggestion given us during our Major project work.

We heartily extend my words of gratitude to Assistant Professor **Mrs. Swaroopa K**, Department Major Project coordinator for her valuable advice, encouragement and suggestion given us during our Major Project work.

We heartily extend my words of gratitude to Sr. Professor **Dr. M.A. Dorairangaswamy**, Institute Major Project coordinator for his valuable advice, encouragement and suggestion given us during our Major Project work.

We would like to express our immense gratitude to Professor **Dr. D Jerline Sheebha Anni**, Head of the Department, CSE for her unfailing encouragement and suggestion given to us in course of Major Project work.

We would like to take opportunity to express our gratitude to Professor **Dr. H S Nanda**, Principal for giving us this opportunity to enrich our knowledge.

We are grateful to the President **Dr. Prabhakara Reddy A** for having provided us with a great infrastructure and well- furnished labs.

Finally, a note of thanks to all the teaching and non – teaching staff of Department of Computer Science and Engineering, for their co-operation extended to us.

Last but not the least, we acknowledge the support and feedback of our parents, guardians and friends, for their indispensable help always.

GAURAV CHAUHAN **1BH23CS024**

GAURAV SINGH **1BH23CS025**

ABSTRACT

VeerDrishti is an Android-based computer-vision prototype for real-time visual situational awareness. It captures camera frames and applies on-device YOLO26 Nano task variants through Flutter YOLOView, with CameraX handling capture and rotation. The prototype displays detection boxes, segmentation masks, pose keypoints, configurable confidence and IoU thresholds, and runtime indicators in portrait and landscape views. The available demonstrations verify the capture-to-display pipeline and task-specific visual outputs. Displayed readings range from 11.9 to 21.3 FPS across the supplied screenshots; these observations are not a sustained accuracy or latency benchmark. Controlled evaluation on annotated, combat-relevant data remains future work.

VISION AND MISSION OF THE INSTITUTE

BANGALORE TECHNOLOGICAL INSTITUTE

VISION

To evolve as a Centre of Excellence in Academics, Research and Development to impart Technological advancement to the Society.

MISSION

M1 - To provide young prospective engineers with an environment for higher education and research

M2 - To mold students as responsible citizens with an attitude of using their knowledge for technological advancements

M3 - To uplift the downtrodden and improve the quality of life of fellow human beings.

VISION AND MISSION OF THE PROGRAM

COMPUTER SCIENCE AND ENGINEERING

VISION

To impart the best in academia & research, empowering the students of Computer Science and Engineering by contributing their best for the society.

MISSION

M1 - To impart updated theoretical and practical knowledge in computer science and engineering by creating an outstanding learning environment.

M2 - To equip the students with wisdom in the discipline of computing and to apply their knowledge for benefiting the society.

M3 - To enhance research collaborations for upliftment of computing professionals.

PROGRAM EDUCATIONAL OBJECTIVES

PEO1 - To showcase strong technical proficiency in computer science fundamentals, empowering the design, development and implementation of innovative solutions that address real world challenges.

PEO2 - To establish a solid foundation in the core concepts of computer science and engineering, while fostering a proactive attitude towards pursuing higher education and fulfilling industry demands.

PEO3 - To embody professional ethics, strong communication skills, and leadership qualities that faster multidisciplinary excellence and career growth.

PEO4 - To cultivate technical knowledge and research ability that inspire innovation, enabling the establishment of startups and fostering entrepreneurial success.

PEO5 - To tackle societal, environmental and global challenges through the design of sustainable and efficient computer science and engineering-based systems that improve overall quality of life

PROGRAM OUTCOMES

1 **Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

2 **Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

3 **Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

4 **Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

5 **Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.

6 **The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

7 **Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

8 **Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

9 **Individual and team work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

10 **Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

11 **Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in environments.

12 **Life-long learning:** Recognize the need for and have the preparation and ability to engage in independent and life-long learning in the context of technological change.

PROGRAM SPECIFIC OUTCOMES

PSO1 - Demonstrate proficiency in computer education by applying principles of mathematics, science & engineering along with technical expertise.

PSO2 - Equip proficiency in computer science and engineering employing the principles of communication networks, database management systems, computing systems & standard software development practices to cater technical growth.

PSO3 - Develop proficiency in analytical skills, research & innovation contributing effectively as per the industry requirements by delivering effective socio - economic skills for the advancement in technology.

INDEX

ACKNOWLEDGEMENT	**i**
ABSTRACT	**ii**
VISION AND MISSION OF THE INSTITUTE	**iii**
VISION AND MISSION & PEO OF THE PROGRAM	**iv**
PROGRAM OUTCOMES	**v**
PROGRAM SPECIFIC OUTCOMES	**vi**
TABLE OF CONTENTS	**vii–viii**
LIST OF FIGURES	**ix**
LIST OF TABLES	**x**
LIST OF SNAPS	**xi**

TABLE OF CONTENTS

Chapter No.	CHAPTERS	Page No.
1	**INTRODUCTION**	**1–5**
	1.1 Overview	1
	1.2 Problem Statement	1
	1.3 Motivation	2
	1.4 Objectives	2–3
	1.5 Methodology	3
	1.6 Sustainable Development Goal Relevance	3–4
	1.7 Feasibility Study	4
	1.8 Work Distribution	5
2	**LITERATURE SURVEY**	**6–14**
	2.1 Literature Review	6–10
	2.2 Comparative Study Of References	11–14
3	**REQUIREMENTS AND ANALYSIS**	**15–17**

	3.1 Functional Requirements	15
	3.2 Non-Functional Requirements	15–16
	3.3 Hardware Requirements	16
	3.4 Software Requirements	16
	3.5 Acceptance Criteria	17
4	**SYSTEM DESIGN**	**18–19**
	4.1 System Architecture	18
	4.2 System Workflow	18–19
5	**SCHEDULING**	**20–21**
	5.1 Gantt Chart	20
	5.2 Month-wise Progress	20–21
6	**IMPLEMENTATION**	**22–29**
	6.1 Prototype Overview	22
	6.2 Module Implementation	22–24
	6.3 Frame Processing Pipeline	24
	6.4 Interface and Detection Controls	25
	6.5 Integration and Testing	25–29
7	**RESULT**	**30–37**
	7.1 Demonstration Setup	30
	7.2 Visual Output	31–33
	7.3 Displayed Runtime Measurements	33–35
	7.4 Discussion and Limitations	36–37
8	**CONCLUSION**	**38**
	REFERENCES	**39–40**
	ICSTIM 2026 ABSTRACT SUBMISSION	**41**
	DRDO SYNOPSIS SUBMISSION	**42**
	APPENDIX A	**43–45**
	APPENDIX B	**46**

LIST OF FIGURES

Fig. No.	Title of the Figure	Page No.
4.1	System Architecture	18
4.2	System Workflow	19
5.1	Gantt Chart for Major Project	20

LIST OF TABLES

Table No.	Title of the Table	Page No.
1.1	Work Distribution Among Team Members	5
2.1	Comparison Table of Research Papers	11–14
7.1	Displayed Runtime Readings	34

LIST OF SNAPS

Snap No.	Title of the Snap	Page No.
7.1	Detection, Segmentation, and Pose	31
7.2	Landscape Object Detection	32
7.3	Indoor Segmentation Output	32
7.4	Multi-Person Pose Output	33
A-i	ICSTIM 2026 Abstract Submission	41
B-i	DRDO Synopsis Submission	42

Chapter 1

INTRODUCTION

1.1 Overview

Modern warfare has transitioned into an increasingly data-driven environment where split-second decisions are critical for survival and mission success. The sheer volume of visual information from surveillance feeds, drones, and tactical environments often overwhelms personnel, leading to cognitive fatigue and a higher margin for error. Manual threat assessment is no longer sufficient to ensure the safety and tactical superiority of ground units in high-stress scenarios. Consequently, there is an urgent need for intelligent, autonomous systems that can process vast amounts of real-time visual data to provide actionable insights and enhance situational awareness.

VeerDrishti is an AI-enabled combat vision system designed to bridge this gap by providing soldiers with a persistent, intelligent observer through a camera-integrated solution. By leveraging computer vision models and edge computing, the system aims to highlight people, vehicles, and potentially relevant objects. The primary objective is to offload the mental burden of constant monitoring from the soldier, allowing them to focus on strategic execution. Developed as a lightweight solution suitable for deployment on wearable hardware, the system aims to transform how visual data is utilized on the front lines of national security.

The current work develops the project into an Android prototype with live video capture, on-device model execution, and visual overlays. The demonstrated prototype shows detection of everyday objects, segmentation, and human pose visualization using CameraX, YOLO26 Nano task variants, and Flutter. The literature review explores approaches such as synthetic data augmentation, visible–thermal fusion, and vision-language detection. These remain potential extensions; the demonstrated prototype establishes an RGB-camera processing pipeline rather than a completed multimodal or combat-validated system.

1.2 Problem Statement

The problem statement is to address the challenges of managing vast real-time visual data in modern battlefield environments, where manual monitoring is slow, error-prone, and inefficient; in order to enhance situational awareness and tactical decision-making, an AI-powered combat vision system is proposed to automatically detect, classify, and assess threats in real time, improving response speed and operational efficiency.

1.3 Motivation

The motivation for VeerDrishti is to minimize casualties and save lives by providing an intelligent, persistent observer in combat environments. Modern warfare involves overwhelming visual data that leads to cognitive fatigue, increasing the risk of fatal human error. The system serves as a technological safeguard, enhancing personnel safety and protecting national security forces during high-stress operations.

Scope

The scope of the project is to develop an autonomous, real-time vision system designed to detect people, weapons, and vehicles in tactical environments. The current work focuses on the Android camera-to-overlay pipeline and ordinary object, segmentation, and pose demonstrations. Identifying hostile intent and validating combat-specific detection require additional data and evaluation.

Purpose

The purpose of the project is to provide an autonomous, intelligent observer that delivers real-time situational awareness in high-stress combat environments. By automating threat detection and mitigating cognitive fatigue, the system serves as a technological safeguard dedicated to achieving the project's core mission: minimizing casualties and saving lives.

Applicability

The project will be applicable in active combat operations, border surveillance, and high-risk tactical missions where ground units require instantaneous threat identification. The edge-optimized architecture will ensure reliability in signal-denied zones and low-visibility environments, serving as a critical force multiplier for national security personnel.

Beyond defense, the system's ability to automate hazard detection will be directly transferable to emergency response and disaster management, fulfilling the mandate to minimize casualties and save lives.

1.4 Objectives

The project's primary objectives are to develop a lightweight, real-time combat vision system that can be deployed on wearable hardware, capable of autonomously detecting and classifying threats in dynamic battlefield environments.

Specific objectives for the project include:

i. Implement a lightweight real-time object detection model.

ii. Enable on-screen visual highlighting of identified threats.

iii. Optimize system performance for low latency and smooth operation.

iv. Evaluate accuracy and response time under simulated field conditions.

1.5 Methodology

The current methodology follows an on-device capture–inference–visualization pipeline. Android CameraX captures live frames, and the native YOLO pipeline handles rotation, resizing or letterboxing, RGB conversion, and tensor preparation. YOLO26 Nano task variants perform model execution on the device. Flutter YOLOView presents bounding boxes, labels, confidence values, segmentation regions, or pose keypoints over the preview, while Logcat supports development diagnostics. Users can adjust detection thresholds and inspect displayed runtime indicators.

The demonstration screenshots provide initial functional evidence. Further evaluation requires a documented device and model configuration, annotated test data, and repeatable timing runs. Combat-specific fine-tuning is part of the planned development. GAN augmentation, multimodal fusion, and open-vocabulary recognition are research-informed options rather than features demonstrated by the current prototype.

1.6 Sustainable Development Goal Relevance

The project aligns with United Nations Sustainable Development Goal 16: Peace, Justice, and Strong Institutions.

Target 16.1: Significantly reduce all forms of violence and related death rates everywhere

VeerDrishti will enhance battlefield situational awareness by detecting and tracking potential threats in real time. This capability will enable defense personnel to respond faster and make more informed decisions during combat situations. By improving awareness and reaction time, the system will help reduce operational risks and potentially decrease casualties.

Target 16.3: Promote the rule of law at the national and international levels and ensure equal access to justice for all

VeerDrishti will record visual data during operations, enabling improved mission analysis and operational evaluation. It will support evidence-based evaluation of decisions made

during missions. Such recorded data could also assist investigations related to violations of international humanitarian law, including potential war crimes.

Target 16.A: Strengthen relevant national institutions, including through international cooperation, for building capacity at all levels, in particular in developing countries, to prevent violence and combat terrorism and crime

VeerDrishti will integrate AI-based detection and tracking technologies, strengthening the technological capacity of national security institutions. The project will support peacekeeping operations and the protection of national interests.

1.7 Feasibility Study

The feasibility of the proposed combat vision system is assessed across technical, operational, and economic dimensions.

Technical Feasibility

The proposed system is technically feasible using existing smartphone hardware and computer vision frameworks. Modern phones provide high-resolution cameras, sufficient processing power, and GPU support for real-time object detection. Lightweight AI models can be optimized for edge deployment, enabling on-device threat detection without requiring complex external hardware.

Operational Feasibility

The system can be integrated into existing helmet setups with minimal modification, using a mounted smartphone as both camera and display interface. It reduces cognitive load by automating threat detection and visual highlighting. Basic training would enable personnel to interpret AR overlays effectively in dynamic environments.

Economic Feasibility

Using commercially available smartphones significantly lowers development and deployment costs compared to specialized military hardware. Open-source AI frameworks further reduce software expenses. The prototype approach minimizes initial investment, making the system cost-effective for research, testing, and potential scalable improvements in future versions.

1.8 Work Distribution

The project tasks were distributed between the team members according to their responsibilities in model development, interface implementation, testing, presentation, and documentation.

Table 1.1: Work Distribution Among Team Members

Team Member	**Assigned Responsibilities**
Gaurav Chauhan	Dataset preparation, annotation, model training and evaluation; system design, module integration, camera-feed rotation handling; requirements analysis, literature review, feasibility study, report preparation, presentation, proofreading, and project demonstration.
Gaurav Singh	User-interface implementation, camera integration, real-time display controls, functional testing and performance observation; requirements analysis, literature review, feasibility study, report preparation, presentation, proofreading, and project demonstration.

Chapter 2
LITERATURE SURVEY

2.1 Literature Review

X. Zhuang et al. presents an innovative approach to military target detection by combining EfficientDet with a Generative Adversarial Network to improve training data quality and detection accuracy. Initially, EfficientDet is applied to identify targets within the original dataset. Subsequently, the GAN generates synthetic military target images, which are incorporated into the training dataset to enhance its diversity and robustness. The expanded dataset is then used to retrain the EfficientDet model, leading to significantly improved detection performance, particularly in situations where limited training data is available. Experimental evaluation on a custom military target dataset demonstrated the effectiveness of the proposed method. The model achieved a mean Average Precision (mAP) of 92.8%, surpassing the performance of conventional approaches such as YOLOv3 and YOLOv5. Moreover, the system maintained real-time processing capability at 15 FPS and achieved over 90% detection accuracy under challenging conditions, including dim lighting and occlusions **[1]**.

C. Konar et al. presents a portable and edge-optimized surveillance system developed to detect modern security threats such as concealed weapons and drones. The proposed framework is designed for deployment in resource-constrained environments, including remote border regions and civilian infrastructure, while maintaining military-grade object detection capabilities. The system is built upon the Florence v2 Vision-Language Transformer and incorporates Parameter Efficient Fine-Tuning (PEFT) along with Low-Rank Adaptation to improve adaptation efficiency with reduced computational overhead. The model was trained using a dataset of 12,000 images, including GAN-generated synthetic samples to enhance detection robustness. For deployment, the framework utilized a Raspberry Pi 5 integrated with an ESP32-S2 module, enabling wireless alert transmission through the ESP-NOW communication protocol. Experimental results demonstrated the effectiveness of the proposed system, achieving an accuracy of 88.01% and outperforming YOLOv8 and Faster R-CNN. Additionally, the model attained a mAP_{50} score of 64.83 while maintaining high efficiency on edge hardware with low CPU and memory utilization **[2]**.

A. A. S. Matta et al. presents an optimized spatiotemporal deep learning framework for predictive behavioral threat detection in surveillance footage. The proposed system addresses challenges in anomaly detection, including complex human behavior, temporal variability, and limited annotated data, by integrating a Convolutional Neural Network with Long Short-Term Memory networks. The CNN component extracts spatial features from video frames, while the LSTM captures temporal dependencies across sequential frames,

enabling effective recognition of suspicious activities. The framework also incorporates hyperparameter optimization, regularization techniques, and cross-validation strategies to improve model stability and generalization. Experimental evaluation was conducted using the DCSASS surveillance dataset containing multiple anomaly categories such as robbery, assault, and vandalism. Results demonstrated that the optimized CNN-LSTM model achieved an impressive accuracy of 98.1%, with high precision, recall, and F1-scores across 3-fold, 5-fold, and 10-fold validation settings. Furthermore, the proposed framework outperformed traditional machine learning models and recent deep learning approaches, proving highly effective for real-time intelligent surveillance applications **[3]**.

N. Li et al. proposes PDGV-DETR, an advanced framework for precise weapon and personnel detection in static security surveillance images. The study addresses critical challenges such as object occlusion and complex backgrounds, which often reduce the effectiveness of conventional detection models in public safety applications, including school and subway surveillance systems. The proposed framework enhances detection performance through three major components. First, a dynamic hierarchical channel interaction convolution module improves the recognition of partially occluded objects. Second, an improved bidirectional hybrid feature pyramid network enables effective cross-scale feature fusion for detecting objects of varying sizes. Finally, a global semantic weaving network strengthens the model's ability to distinguish targets from cluttered and complex backgrounds in static scenes. Experimental evaluation demonstrated the effectiveness of PDGV-DETR across multiple datasets. The framework achieved a peak mAP_{50} of 85.9% on conflict-scene datasets and 93% on weapon-specific datasets. Furthermore, the model significantly outperformed RT-DETR and Deformable DETR, while maintaining a balance between high localization accuracy and computational efficiency suitable for real-time surveillance applications **[4]**.

A. M. Garcia-Rubio et al. presents a collaborative AI-human framework for detecting concept drift in object detection models used in defense and surveillance applications. The proposed system addresses the degradation of object detection performance caused by evolving environments, lighting conditions, and unfamiliar objects. The framework integrates a YOLOv7 architecture with human feedback to improve reliability and adaptability in dynamic military scenarios. A custom metric called the Drift Avoidance Value was introduced to quantify concept drift by combining AI-generated confidence scores with human corrections, duplicate detection analysis, and missed object evaluations. The model was trained on 4,579 military vehicle images and evaluated using multiple datasets containing main battle tanks, humans, and unknown ground vehicles. Experimental results demonstrated that retraining the model with human-corrected data significantly improved detection performance. The Drift Avoidance Value increased after retraining, while prediction accuracy improved by 16.9% for tanks and 12.5% for humans. The study

highlights the effectiveness of human-in-the-loop learning for maintaining robust and adaptive object detection systems in real-world defense environments **[5]**.

X. Xiang et al. introduces UniFusOD, an end-to-end framework designed for infrared–visible image fusion and object detection. The proposed system combines visible images, which provide rich texture and color information, with infrared images that offer strong edge details and reliable performance under low-light conditions. By integrating these complementary modalities, UniFusOD generates a single fused image optimized for both visual clarity and accurate object detection. The framework incorporates the FRA mechanism, which utilizes multiple convolution kernels to capture fine-grained details as well as broader semantic features. Additionally, the proposed UnityGrad Optimization strategy balances fusion and detection gradients, preventing conflicts between the two tasks and enabling more effective joint training. Experimental evaluation demonstrated that UniFusOD outperformed several state-of-the-art methods across multiple benchmark datasets. The model achieved improved mean Average Precision (mAP) scores, including gains of 0.8 on the DroneVehicle Dataset and 1.9 on the M3FD Dataset. Furthermore, the framework produced fused images with superior structural quality and information content while maintaining strong robustness in detecting small targets within complex environments **[6]**.

Z. Shou et al. presents a robust framework for cyber physical systems, including autonomous vehicles and smart surveillance applications, through the development of MSAF-Net for human action recognition. The proposed system integrates multiple visual modalities such as RGB, depth, and thermal images to enhance detection accuracy and reliability in complex environments. The framework employs a Multi-Scale Attention Fusion mechanism that extracts features at different levels of detail while emphasizing the most relevant information for accurate recognition. In addition, Cross-Level Feature Interaction enables different network layers to exchange information, preserving both fine-grained details and global contextual structures. The model also incorporates an Adaptive Fusion Strategy that combines semantic and structural guidance to generate clear and consistent fused representations. Experimental evaluation on several benchmark datasets demonstrated the effectiveness of the proposed approach. MSAF-Net achieved 91.54% accuracy on the UCF101 Dataset and 92.14% accuracy on the ActivityNet, outperforming existing state-of-the-art methods. Furthermore, the framework remained robust even when individual sensors failed and could be efficiently pruned for deployment on low-power hardware platforms **[7]**.

Z. Song et al. introduces EDNet, a deep learning framework designed for detecting small objects in UAV and drone imagery. Built upon YOLOv10, the framework is optimized for edge computing, enabling efficient deployment on mobile and low-power devices.

EDNet is highly scalable and offers seven model variants ranging from Tiny, which is lightweight and computationally efficient, to XL, which provides high-performance detection capabilities. The architecture incorporates the C2f-FCA block, which utilizes faster context attention mechanisms to extract meaningful features while maintaining low computational complexity. In addition, the model employs the WIoUv3 Loss Function to prioritize high-quality samples and reduce the influence of noisy data during training. For optimized deployment, the framework supports technologies such as CoreML, TensorRT, and OpenVINO, ensuring high-speed performance across multiple hardware platforms. Experimental results demonstrated that EDNet outperformed YOLOv8 and YOLOv10 in both accuracy and efficiency, achieving up to 5.6% higher mAP_{50} with fewer parameters while maintaining real-time performance across devices such as the iPhone 12 and Raspberry Pi 5 **[8]**.

S. D N et al. presents a state-of-the-art object detection system designed for analyzing camera and drone footage in military environments. The proposed framework focuses on identifying battlefield threats such as vehicles, weapons, and combatants in real time. The system employs a vision-language model that combines image data with natural language descriptions to improve object understanding and classification accuracy. It integrates CLIP to associate images with textual descriptions, enabling the detection of unseen objects without requiring extensive labeled datasets. Additionally, the SPPELAN pooling module is used to generate fixed feature representations while minimizing information loss. The framework also incorporates an N-gram and GLIP labeler to extract noun phrases and create pseudo-boundaries for automated training data generation. Furthermore, K-means Clustering is applied to group similar image regions, improving detection efficiency in cluttered environments. Experimental evaluation on the Microsoft COCO Dataset demonstrated superior performance compared to Grounding DINO, achieving lower complexity, higher accuracy, and significantly faster frame rates suitable for real-time deployment **[9]**.

V. Kaya et al. proposes a deep learning-based approach for the automatic detection and classification of seven different handheld weapon types. The study aims to assist security agencies by developing a proactive surveillance system capable of identifying threatening objects in real time, thereby helping to prevent criminal activities. The proposed model is based on the VGGNet architecture and consists of 25 layers with 337,671 parameters, making it both efficient and computationally lightweight. A custom dataset containing 5,214 images of various weapons, including assault rifles, pistols, and grenades, was collected from internet sources and preprocessed using grayscale conversion and background cleaning techniques. The model was implemented and trained using the Keras deep learning framework. Experimental results demonstrated that the proposed CNN model achieved an impressive classification accuracy of 98.40%, significantly

outperforming models such as VGG-16, ResNet50, and ResNet-101. Additionally, the model achieved a low loss rate of 0.052, highlighting its reliability, fast training capability, and suitability for deployment on low-cost computing systems **[10]**.

Ankita et al. introduces an efficient and lightweight deep learning framework for human activity recognition using smartphone sensor data. The study addresses the limitations of traditional pattern recognition techniques by employing automated feature extraction, which is highly beneficial for applications such as healthcare, elderly assistance, and sports science. The proposed model integrates a Convolutional Neural Network with Long Short-Term Memory units to form a CNN-LSTM architecture capable of learning both spatial and temporal features. Convolutional layers automatically extract meaningful patterns from accelerometer and gyroscope signals, while LSTM layers analyze sequential dependencies in human activities. The model was trained and evaluated using the UCI-HAR Dataset collected from Samsung Galaxy S2 smartphones, with Gaussian standardization applied during preprocessing. Experimental results demonstrated that the CNN-LSTM model achieved an impressive recognition accuracy of 97.89%. Furthermore, the model outperformed traditional machine learning approaches by effectively recognizing activities such as walking, sitting, and standing while maintaining a lightweight structure suitable for mobile deployment **[11]**.

2.2 Comparative Study Of References

Table 2.1: Comparison Table of Research Papers

Sl. No.	Research Paper	Learning Outcome
1	**Title:** Military target detection method based on EfficientDet and Generative Adversarial Network **Authors:** X. Zhuang, D. Li, Y. Wang, and K. Li **Published:** Engineering Applications of Artificial Intelligence, vol. 132, Jun. 2024	**Objective:** To detect targets in low visibility, noisy, or data-scarce combat zones **Methodology:** EfficientDet-D0 + Dual-Discriminator GAN **Core Idea:** Uses GANs to generate synthetic military training data and BiFPN for feature fusion
2	**Title:** Edge-Optimized Threat Detection with Florence v2 **Authors:** C. Konar, D. Das, D. S., K. Jc, and S. Singh **Published:** PeerJ Computer Science, vol. 12, Mar. 2026	**Objective:** To create a portable edge-optimized surveillance system capable of detecting military-relevant threat classes in resource-constrained environments **Methodology:** Florence v2 Vision-Language Transformer **Core Idea:** Leveraging multimodal vision-language models on compact edge hardware to provide situational awareness without relying on cloud infrastructure
3	**Title:** Spatiotemporal deep learning framework for predictive behavioral threat detection in surveillance footage **Authors:** A. A. S. Matta and V. P. C. S. R. Manukonda **Published:** Frontiers in Big Data, vol. 9, Feb. 2026	**Objective:** To detect anomalous human behaviors in untrimmed surveillance footage using optimized spatiotemporal deep learning **Methodology:** Hybrid CNN-LSTM framework on DCSASS dataset **Core Idea:** Integrating CNN spatial features and LSTM temporal patterns to reliably identify 13 distinct types of criminal behavior

Continued on next page

Sl. No.	Research Paper	Learning Outcome
4	**Title:** PDGV-DETR: Object Detection for Secure On-Site Weapon and Personnel Location **Authors:** N. Li et al. **Published:** Sensors, vol. 26, Feb. 2026	**Objective:** To precisely detect and position weapons and personnel in static surveillance images while overcoming occlusion and complex background interference **Methodology:** PDGV-DETR framework **Core Idea:** Transformer-based end-to-end detection system that uses cross-scale semantic fusion to bridge the gap between high-level semantics and low-level image details
5	**Title:** Detecting Concept Drift in Object Detection Models: A Collaborative AI-Human Approach for Defense Applications **Authors:** A. M. Garcia-Rubio, L. Heck, K. M. Schweitzer, R. M. Bateman, A. Alaeddini, and K. K. Castillo-Villar **Published:** IEEE Access, vol. 14, Jan. 2026	**Objective:** To quantify and manage model performance deterioration in military object detection systems over time **Methodology:** YOLO confidence scores + direct human feedback **Core Idea:** Enhancing AI reliability by integrating human corrections directly into drift calculations to guide targeted and efficient model retraining
6	**Title:** Infrared-Visible Image Fusion Meets Object Detection **Authors:** X. Xiang et al. **Published:** Remote Sensing, vol. 17, Nov. 2025	**Objective:** To solve gradient conflicts when training fusion and detection together **Methodology:** UniFusOD **Core Idea:** Use Fine-Grained Region Attention (FRA) mechanism and a unified framework to fuse RGB and infrared images for accurate detection

Continued on next page

Sl. No.	Research Paper	Learning Outcome
7	**Title:** Multi-modal action recognition via advanced image fusion techniques **Authors:** Z. Shou and D. Zhu **Published:** Frontiers in Physics, Aug. 2025	**Objective:** To recognize human or combatant actions accurately even if one sensor fails **Methodology:** MSAF-Net (Multi-Scale Attention-Guided Fusion) **Core Idea:** Dynamically prioritizes sensor data and switches to backup sensors when needed
8	**Title:** EDNet: Edge-Optimized Small Target Detection in UAV Imagery **Authors:** Z. Song, Y. Zhang, and A. A. R. M. A. Ebayyeh **Published:** IEEE SWC, Dec. 2024	**Objective:** To enable high-speed detection of tiny objects on low-power drone hardware **Methodology:** YOLOv10 + Faster Context Attention (C2f-FCA) **Core Idea:** Uses an XSmall detection layer for small object detection with hardware acceleration optimization
9	**Title:** Real-Time Versatile Object Detection With High Accuracy And Reduced Information Loss During Wartime **Authors:** S. D. N. and D. M. **Published:** IEEE Conference, Sep. 2024	**Objective:** To detect unseen or new battlefield threats without manual re-labeling **Methodology:** CLIP Encoder + Modified C2f-Darknet **Core Idea:** Uses pseudo-boundary labeling for zero-shot detection and combines vision and language

Continued on next page

Sl. No.	Research Paper	Learning Outcome
10	**Title:** Detection and Classification of Different Weapon Types Using Deep Learning **Authors:** V. Kaya, S. Tuncer, and A. Baran **Published:** Applied Sciences, vol. 11, Aug. 2021	**Objective:** To propose a model for the automatic detection and classification of distinct types of handheld weapons **Methodology:** CNN based on VGGNet architecture **Core Idea:** Utilizing reduced layer complexity and high-precision CNNs to enable weapon identification on low-cost computing systems
11	**Title:** An Efficient and Lightweight Deep Learning Model for Human Activity Recognition Using Smartphones **Authors:** Ankita, S. Rani, H. Babbar, S. Coleman, A. Singh, and H. M. Aljahdali **Published:** Sensors, vol. 21, Jun. 2021	**Objective:** To develop an efficient and lightweight model for identifying human activities using smartphone-based sensor data **Methodology:** CNN + Long Short-Term Memory (LSTM) **Core Idea:** Integrating spatial and temporal deep learning architectures to automate feature extraction from raw sensor data

Chapter 3
REQUIREMENTS AND ANALYSIS

The requirements analysis for the VeerDrishti combat vision system encompasses a comprehensive evaluation of functional and non-functional requirements, ensuring that the system meets the operational needs of modern warfare while adhering to technical constraints. The analysis is structured to identify key features, performance metrics, and user interactions necessary for the successful deployment of the system in dynamic battlefield environments.

3.1 Functional Requirements

The functional requirements define the core capabilities of the project, including:

i. **Real-Time Object Detection:** The system must be capable of detecting and classifying combatants, vehicles, and weapons in real time using the integrated camera feed.

ii. **Visual Highlighting:** Detected threats should be visually highlighted on the display interface, providing clear and immediate feedback to the user.

iii. **Open Vocabulary Recognition:** The system should be able to identify novel or previously unseen threats using natural language supervision and language-vision encoders as a future extension; the current task variants use their configured class sets.

iv. **User Interface:** The system must provide an intuitive user interface that allows easy interpretation of the visual information to make informed decisions without distraction.

v. **Data Logging:** The system must provide secure local storage of detection events and video metadata for post-mission analysis and system auditing in a later implementation stage. The present prototype records development diagnostics through Logcat rather than persistent mission storage.

3.2 Non-Functional Requirements

The non-functional requirements address the performance, reliability, and usability aspects of the project, including:

i. **Low Latency:** The system must operate with minimal latency to ensure timely threat detection and response in high-stress combat scenarios.

ii. **Robustness:** The system should maintain high detection accuracy under varying environmental conditions, such as low light, occlusion, and dynamic backgrounds.

iii. **Scalability:** The system should be designed to accommodate future enhancements, such as additional threat categories or integration with other battlefield systems.

iv. **Security:** The system must ensure the confidentiality and integrity of the data collected, preventing unauthorized access and ensuring compliance with military data handling standards.

v. **Usability:** The system should be user-friendly, requiring minimal training for effective operation, and should not contribute to cognitive overload for the user.

3.3 Hardware Requirements

i. **Development Workstation:** High-performance laptop equipped with a dedicated GPU for code development, dataset preprocessing, and initial model debugging.

ii. **Edge Deployment Device:** Android smartphone with a camera and sufficient compute and memory to execute the deployed model and render a live preview. The exact device and acceleration backend must be recorded for reproducible performance measurements.

iii. **Training Resources:** GPU-equipped local or cloud resources for planned model fine-tuning. The final deployment record should identify the training resource and completed configuration used for reproducible experiments.

3.4 Software Requirements

i. **Target Platform:** Android with camera permission, CameraX capture, and a native inference integration. The Android version and minimum SDK must match the application build.

ii. **Application Framework:** Flutter and Dart for the user interface and YOLOView integration, together with the Android native components used by the camera and model runtime.

iii. **Model Pipeline:** Ultralytics YOLO26 Nano and task variants for on-device detection, segmentation, and pose visualization.

iv. **Development and Diagnostics:** A Flutter/Android development environment, Android SDK tools, and Logcat for runtime diagnostics. Record actual package versions when preparing the final reproducible setup.

v. **Model Preparation:** A compatible Ultralytics training/export environment for custom fine-tuning. The dataset, export format, and framework versions must be recorded with any completed training run.

3.5 Acceptance Criteria

The requirements are considered traceable when each implemented capability can be checked through an observable system action and a recorded test condition. Functional acceptance checks include camera initialization, frame processing, task selection, overlay rendering, threshold control, and graceful handling of camera or model errors. Non-functional checks should record the device, application build, model variant, input configuration, processing time, displayed frame rate, and operating conditions. This separation distinguishes interface verification from accuracy, robustness, power, and sustained-performance evaluation.

Chapter 4
SYSTEM DESIGN

4.1 System Architecture

The system architecture connects live video capture, preprocessing, AI detection, post-processing, overlay visualization, and the user interface. The architecture overview is shown in Figure 4.1. The layered diagram describes how visual input is transformed into information displayed to the user.

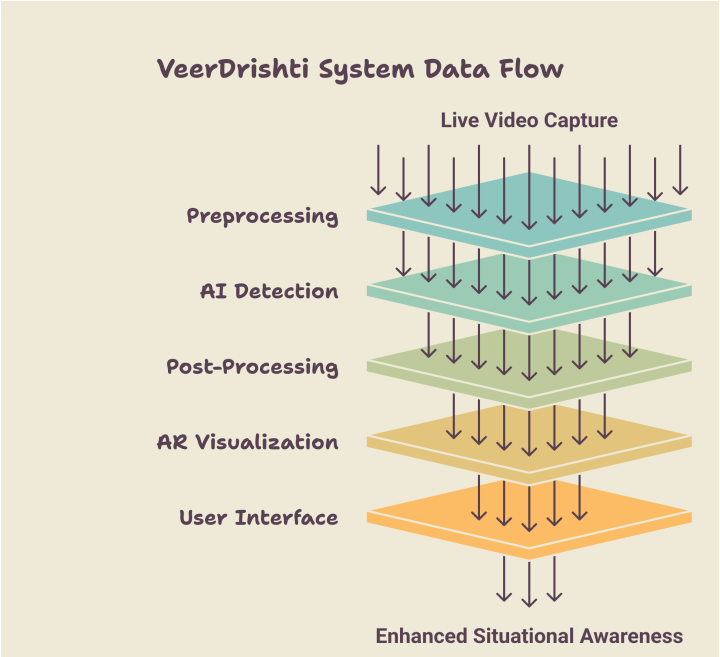


Figure 4.1: System Architecture

In the prototype, CameraX supplies frames, the native YOLO pipeline prepares and analyzes them, and Flutter YOLOView renders the outputs. The design uses the term AR visualization for the overlays; the demonstrated outputs show image-space boxes, masks, and keypoints rather than a world-registered augmented-reality system. Logcat supports development diagnostics independently of the visual pipeline. Persistent mission storage is a proposed extension, not a demonstrated layer of the current implementation.

4.2 System Workflow

The operational workflow is presented in Figure 4.2. The sequence starts with RGB camera input and proceeds through preprocessing, model execution, post-processing, user-interface visualization, and interpretation of the displayed output.

The implemented preprocessing operations are rotation, resizing or letterboxing, RGB conversion, and tensor preparation. After inference, filtering settings and coordinate mapping determine which outputs are drawn on the preview. The detection, segmentation, and pose modes provide different visual representations within this loop. The diagram's

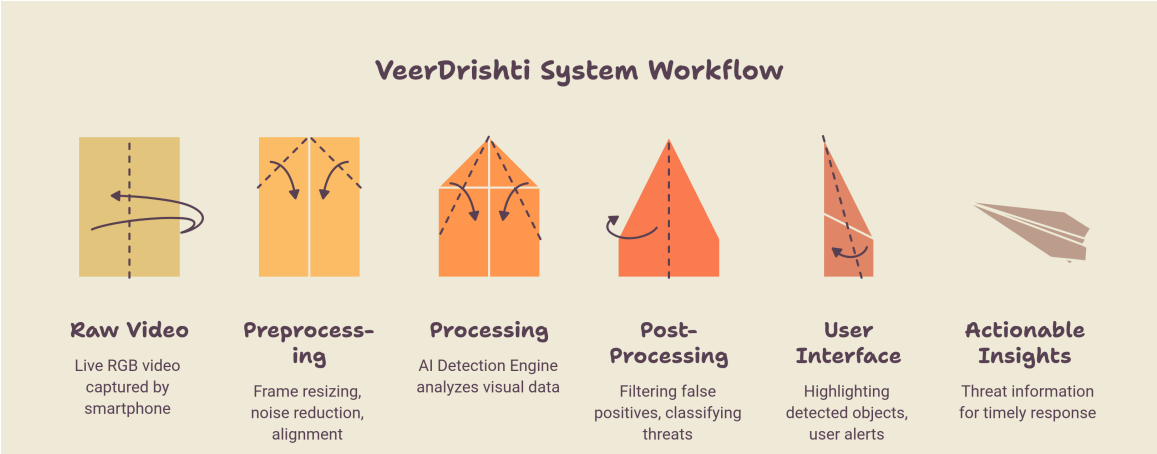


Figure 4.2: System Workflow

references to threat classification and user alerts express the wider design objective; the demonstrated outputs establish object-class overlays and do not demonstrate hostile-intent classification or a separate alerting service.

Chapter 5
SCHEDULING

5.1 Gantt Chart

The development schedule spans July–November 2026. It organizes prototype planning, implementation, interface testing, custom model fine-tuning, model testing, final refinements, and report drafting. The planned sequence is shown in Figure 5.1 in the report's editable Gantt format. Week positions are planning estimates rather than exact start and finish dates.

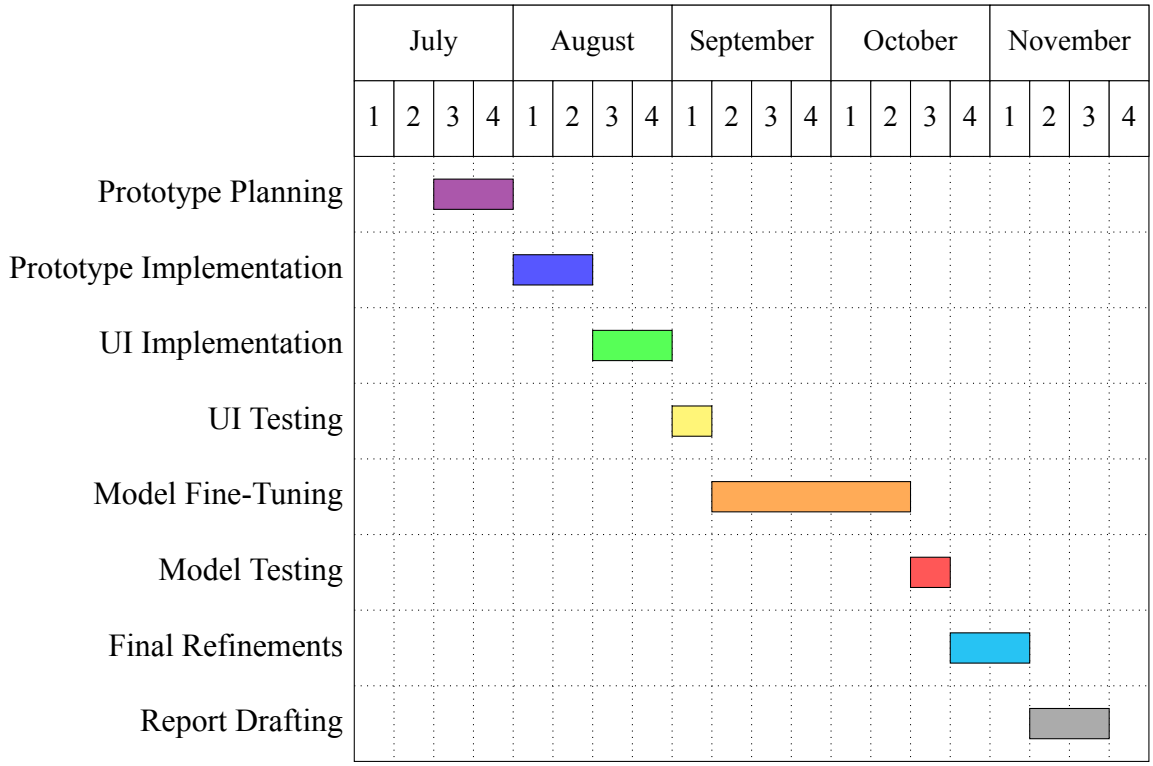


Figure 5.1: Gantt Chart for Major Project

The schedule is a development plan, not evidence that every later task has been completed. The academic year of the final submission is 2026–2027, and the development months shown here retain the planned 2026 dates.

5.2 Month-wise Progress

July

The plan begins with prototype planning in the latter part of July. This stage establishes the mobile processing pipeline and prepares the transition from the earlier literature and design work to an Android implementation.

August

Prototype implementation and UI implementation occupy August in the schedule. The September demonstration provides evidence of an integrated camera view, model outputs, task controls, and visual overlays following these development activities.

September

The schedule places UI testing near the start of September, followed by custom model fine-tuning. The available demonstration includes prototype screenshots but does not provide a completed custom-training run or validation metrics.

October

The plan continues fine-tuning into October, followed by model testing and final refinements. Completion of these tasks should be documented with the model configuration, training records, test data, and measurements produced during implementation.

November

Final refinements and report drafting extend into November. The final report consolidates the implementation details, evaluated results, limitations, and conclusions. Planned tasks are to be updated with actual progress before submission.

Chapter 6
IMPLEMENTATION

6.1 Prototype Overview

VeerDrishti is implemented as an Android application that combines a live camera feed with on-device computer-vision inference. The implementation uses Android CameraX for video capture, Ultralytics YOLO26 Nano and its variants for inference, Flutter for the interface, and Android Logcat for development diagnostics. The prototype brings these components together to display detections directly over the camera view.

The implementation focuses on the capture–inference–visualization loop. The prototype demonstrates ordinary object detection, segmentation overlays, and human pose visualization. The longer-term combat-vision objective guides the application, but thermal fusion, open-vocabulary recognition, combat-specific fine-tuning, and persistent mission recording are separate extensions whose completion is not established by the current prototype evidence.

6.2 Module Implementation

Input Module

Android CameraX provides the live camera input. The application exposes the camera view through the Flutter interface so that users can observe the scene and its associated overlays together. The portrait and landscape screenshots demonstrate operation in both orientations. Camera access requires Android camera permission; denial or interruption must be handled before attempting to run the inference pipeline.

Preprocessing Module

Preprocessing is handled by the native YOLO inference pipeline. This stage includes frame rotation, resizing or letterboxing, RGB conversion, and input tensor preparation. Rotation keeps the analyzed image consistent with device orientation. Resizing and letterboxing adapt the camera frame to the model input while preserving the information needed to align output coordinates with the preview. The exact input resolution and normalization settings depend on the deployed model configuration and should be recorded for reproducibility.

Orientation and Coordinate Handling

Camera frames and displayed previews do not always share the same width, height, or orientation. The implementation therefore treats orientation as part of the input-processing contract rather than as a cosmetic display option. A portrait frame may arrive from the camera with rotation metadata, while the preview is rendered after the interface has already applied its own layout transformation. If these transformations are ignored, a detection can be correct in model coordinates but appear shifted, mirrored, or rotated in the user interface.

The processing sequence first reads the camera rotation information, then prepares the frame for the selected task. After inference, the output coordinates are mapped back to the preview dimensions. The same principle applies to segmentation masks and pose keypoints: the transformation must be consistent for the image, the bounding geometry, and the visual overlay. This is why the portrait and landscape demonstrations are useful integration evidence even though they are not accuracy tests.

The current implementation material also indicates that camera-feed rotation is applied consistently when frames are passed through the inference path. A production evaluation should still record the rotation metadata, preview dimensions, crop or letterbox policy, and zoom state for every run. These details make it possible to distinguish an inference problem from a coordinate-mapping problem when an overlay appears misaligned.

Inference Module

YOLO26 Nano and its task variants form the inference component. Computation takes place on the device, supporting a prototype that does not require a remote server for each frame. The interface exposes Det, Seg, and Pose modes, corresponding to detection, segmentation, and pose outputs. The precise model export format, class configuration, and hardware execution delegate must be taken from the application configuration when documenting a reproducible deployment.

Task Selection and Threshold Handling

The task selector changes the type of output produced by the same camera-to-display workflow. Detection emphasizes rectangular object localization and class labels, segmentation adds pixel-region information, and pose adds keypoints and skeletal connections for visible people. Keeping these modes inside one interface makes the prototype suitable for comparing the presentation cost and visual interpretation of different outputs without changing the surrounding camera workflow.

The confidence threshold provides a first filtering boundary for predictions shown to the user, while the IoU threshold affects overlap handling where the selected pipeline applies suppression. A lower confidence threshold can expose more candidate objects but may also increase clutter. A stricter threshold can make the view easier to interpret while hiding weak or partially occluded predictions. For this reason, a threshold value should be treated as part of the experiment configuration, not as a hidden interface preference.

The demonstrated configuration uses a confidence threshold of 0.25 and an IoU threshold of 0.70. Those values explain the visible output in the supplied exhibits, but they do not define an optimal configuration for combat scenes. Future tests should vary one setting at a time, preserve the same scene and model, and record the resulting changes in visible detections, masks, or keypoints.

Post-Processing Module

The native inference output is passed to the Flutter YOLOView integration for display. The application renders bounding boxes, class labels, and confidence information on the live camera feed. Segmentation mode adds colored object-region overlays, while pose mode displays body keypoints and their connecting lines. Detection coordinates must be mapped to the displayed preview after rotation, letterboxing, scaling, or zoom so that the overlays remain aligned with the visible objects.

User Interface Module

Flutter supplies the application interface, camera display, mode controls, and visual output. The demonstration screenshots show confidence and intersection-over-union (IoU) threshold controls, zoom information, and runtime indicators. Keeping these controls in the camera view allows users to inspect the effect of a configuration change without switching to a separate results screen. Object labels describe model predictions; an object or person label alone does not establish hostile intent or distinguish a combatant from a civilian.

Development Logging Module

Android Logcat records runtime logs and debugging information during development and testing. These logs support diagnosis of camera initialization, inference integration, and application behavior. This development logging mechanism is distinct from a persistent detection-event or mission-video archive. The current implementation does not include a demonstrated database, encrypted recording service, or post-mission retrieval interface.

6.3 Frame Processing Pipeline

The module descriptions define the following processing sequence for the prototype:

i. Initialize the selected camera and model task, then present the live preview after camera permission is available.

ii. Obtain an analyzable frame from CameraX and account for its orientation.

iii. Resize or letterbox the frame, convert it to RGB, and prepare the input tensor in the native inference pipeline.

iv. Run the selected YOLO26 Nano task variant on the device and obtain its output.

v. Apply the configured filtering settings and map the resulting coordinates to the preview.

vi. Render labels, boxes, masks, or pose keypoints through YOLOView, display runtime information, and continue with subsequent frames.

6.4 Interface and Detection Controls

The demonstration screenshots show a confidence threshold of 0.25 and an IoU threshold of 0.70. The confidence setting controls the minimum score accepted for display. The IoU setting controls overlap filtering where the selected model pipeline uses it. Changing either threshold can alter the visible output, so evaluation runs should record both values along with the selected task and model. The implementation record should state whether all task variants use the same suppression mechanism.

The interface also displays frames per second and processing times, including preprocessing, inference, and post-processing components. These values help inspect responsiveness during a demonstration. They are displayed runtime observations, not a substitute for repeated timing measurements on a documented device. Chapter 7 records the displayed values and distinguishes them from a sustained-performance benchmark.

6.5 Integration and Testing

Integration testing connects camera capture, model execution, and overlay rendering as one application workflow. The demonstrated exhibits involve scissors, a mouse, furniture, laptops, bottles, and groups of people. These scenes exercise multiple visible objects and the three task modes. Portrait and landscape exhibits also provide evidence for checking preview orientation and output alignment.

The integration boundary can be understood as five connected checkpoints. First, the camera must initialize and deliver a frame after permission is granted. Second, the frame must reach the selected model task with the expected orientation. Third, the inference result must return without blocking the interface. Fourth, the result must be mapped into the preview without visible displacement. Finally, the interface must expose the selected mode, threshold state, and runtime fields clearly enough for a user to interpret the output. A failure at any checkpoint can make the final screenshot appear wrong even when the other modules are functioning correctly.

The visible prototype supports functional checks at this boundary. It demonstrates live preview operation, task switching, overlays, portrait and landscape presentation, and displayed timing values. It does not by itself establish permission recovery after interruption, long-duration thermal stability, memory behavior, or reliable operation across different camera sensors. Those cases belong to a later validation plan and should be tested on documented hardware rather than inferred from one successful demonstration.

For reproducible testing, record the Android device and version, application build, model file, task mode, input resolution, thresholds, and execution backend. Separate functional checks from accuracy and performance evaluation: a correctly rendered box verifies an interface path, while precision, recall, mAP, and sustained frame rate require defined data and measurement procedures. The available prototype material supports an initial functional

demonstration; the Result chapter describes the available observations and the evaluation details still needed.

The recommended implementation checklist is therefore: verify camera permission and initialization; verify rotation handling in both orientations; verify the selected task and threshold values; verify that boxes, masks, and keypoints remain aligned after resizing; verify that runtime readings are visible without covering the scene; and record the device, build, model, and execution backend. Completing this checklist would make a later accuracy or performance study easier to reproduce without changing the application architecture described in this chapter.

Implementation Risks and Mitigations

Several practical risks are relevant to an on-device vision application. Camera permission can be denied, revoked, or interrupted when the activity changes state. The application should respond by showing a clear recovery path rather than attempting inference on an unavailable frame. Similarly, model initialization can fail because of a missing file, an incompatible export, or an unavailable execution delegate. Logging the initialization stage and the selected task makes these failures easier to separate from ordinary frame-processing errors.

Orientation and preview scaling create a second class of risk. A camera frame can be valid while its overlay is visually incorrect if the rotation metadata, crop, zoom, or letterbox dimensions are not propagated consistently. The demonstrated portrait and landscape views provide a practical regression pair: the same visible object can be checked after the device orientation changes, and any systematic displacement can be investigated in the coordinate transformation rather than in the model itself.

Runtime stability is another boundary. A single successful frame does not establish that the preview will remain responsive during a long session. Repeated runs should therefore record frame processing behavior, memory pressure, device temperature, and the selected acceleration path. If performance degrades over time, the investigation should distinguish model execution, image conversion, overlay rendering, and logging overhead. This decomposition follows the modular structure already used by the prototype and avoids attributing every delay to the detector.

Finally, the interface must remain understandable when several objects, masks, or keypoints are visible at once. Labels should not obscure the most important part of the scene, controls should be readable in both orientations, and confidence values should be interpreted as model outputs rather than as verified accuracy. These considerations do not change the current prototype result, but they define the engineering checks needed before the same pipeline could be evaluated in more demanding indoor, outdoor, or tactical conditions.

Runtime State and Error Handling

The capture-to-display path is easier to maintain when its runtime states are explicit. A practical sequence is to begin in an uninitialized state, request or confirm camera permission, initialize the preview, load the selected model, and only then accept frames for inference. During a task change, the interface should retain the last valid preview while the new task is prepared, or show a clear loading state if the pipeline must be recreated. Treating these transitions as states prevents a temporary camera or model condition from being mistaken for a detection failure.

Camera permission is the first recoverable boundary. If permission is denied, the application should explain why the camera is unavailable and provide a route for the user to grant access rather than continuing with a null or stale frame. If the activity is paused or the camera is interrupted, the preview should be released or reattached according to the CameraX lifecycle. The next accepted frame should be checked for valid dimensions, rotation metadata, and an available image buffer before it is passed to preprocessing. These checks are ordinary application safeguards, but they are important for an inference application because a malformed frame can create misleading downstream symptoms.

Model initialization is a separate state from camera initialization. A model file can be absent, incompatible with the selected task, or unable to use the requested execution delegate. The interface should report that condition as an initialization error and retain enough diagnostic information to identify the model path, task, and delegate. A fallback execution path may be considered during later development, but it should be recorded because a change from accelerated to non-accelerated execution would affect runtime readings. The current report therefore treats the execution backend as a required configuration field rather than assuming that all devices use the same path.

Frame processing also needs a controlled response to intermittent failures. A single frame that cannot be decoded should be logged and skipped without terminating the whole preview, while repeated failures should be surfaced as a camera or pipeline warning. Inference exceptions should not leave an old overlay on a new frame without an indication that the result is stale. Clearing or timestamping the overlay is preferable to presenting an earlier prediction as if it belonged to the current image. These behaviours are proposed engineering checks for the next validation pass; the supplied screenshots establish the successful path but do not document every recovery state.

The same principle applies to user-controlled settings. A threshold change should be applied at a defined point in the pipeline and reflected in the visible control. If the selected task does not use a particular setting in the same way as another task, the interface or documentation should make that scope clear. The configuration used for a screenshot should be captured with the output so that a later review can distinguish a changed threshold from a changed model or scene. This small amount of state traceability reduces ambiguity without requiring

persistent mission recording.

Configuration Traceability

Reproducibility depends on recording the conditions under which an output was produced. For each demonstration, the minimum record should include the application build, Android version, device model, camera orientation, preview dimensions, selected task, model file or model version, confidence threshold, IoU threshold, maximum item setting if applicable, and execution backend. The record should also identify whether the frame came from a live camera or a static test image. Without these fields, two screenshots with different labels or timings cannot be compared reliably.

The orientation record should contain both the source rotation and the display orientation. A label such as "portrait" alone is insufficient because a camera sensor can deliver a rotated buffer while the preview uses a different coordinate convention. The crop or letterbox policy should be recorded as well, since the same source frame can produce different overlay geometry when the model input aspect ratio changes. Zoom and digital crop settings are also relevant when small objects are being inspected.

Runtime fields should be copied exactly as displayed and should not be silently converted into a new metric. If the interface reports FPS, processing time, preprocessing time, inference time, and post-processing time, each value should retain its label and unit. The time window, averaging rule, and whether the value excludes camera capture should be documented before a formal performance claim is made. The present report follows this rule by transcribing the displayed values and keeping the interpretation configuration-specific.

This traceability approach also supports debugging. If an overlay is displaced only in landscape orientation, the investigation can compare rotation metadata and preview dimensions while holding the model and thresholds constant. If inference time increases after a task change, the task and model can be compared without confusing the result with a different camera scene. If a frame-rate value changes after a build change, the application build and delegate record provide a starting point for examining the cause. The record therefore serves both future evaluation and ordinary implementation maintenance.

Validation Matrix and Reproducible Procedure

A later validation pass can be organized as a matrix rather than as a collection of unrelated screenshots. The first dimension is task mode: detection, segmentation, and pose. The second is device orientation: portrait and landscape. The third is scene condition: clear objects, multiple objects, partial occlusion, small or distant objects, camera motion, and illumination variation. For every cell, the same model and documented device should be used, with repeated samples collected under the same threshold configuration. Empty cells should be reported as untested rather than inferred from a nearby condition.

The functional part of the matrix asks whether the camera preview initializes, the selected

task loads, the output type is rendered, and the overlay remains aligned after orientation and preview-size changes. The measurement part asks for repeated frame-rate and processing-time samples, together with the exact time window and device state. The accuracy part requires labelled reference data and a defined matching rule before precision, recall, or mAP can be computed. Keeping these three parts separate prevents a successful visual overlay from being treated as evidence for all three.

A reproducible run can follow the sequence below. First, record the device, build, model, task, and threshold configuration. Second, start the application from a known lifecycle state and confirm camera permission. Third, allow the preview to stabilize before collecting observations. Fourth, capture a fixed number of samples for the selected scene and preserve the displayed runtime fields. Fifth, repeat the run after changing only one controlled variable, such as orientation or task mode. Finally, store the observations with enough context to identify the image, configuration, and any failure or dropped frame. This procedure does not create a benchmark by itself; it defines the evidence needed for one.

The matrix should also include negative and recovery cases. A denied permission, unavailable model, unsupported task, invalid frame, interrupted camera session, and overlay mismatch are all meaningful results because they identify the boundary of reliable operation. The correct response is to record the failure and its recovery behaviour, not to omit it from the report. For a safety-sensitive concept, clear failure indication is part of system quality even when no detection metric is involved.

The current prototype material supports the successful functional path and supplies useful examples for the first matrix cells. It does not provide the device configuration, repeated sample count, annotated ground truth, or recovery logs required to complete the full matrix. Stating that boundary keeps the implementation chapter reproducible and allows future measured results to be added without rewriting the existing evidence.

Chapter 7

RESULT

7.1 Demonstration Setup

The results in this chapter are based on the demonstrated Android prototype. The available evidence consists of application screenshots in indoor settings, showing everyday objects and groups of people. The screenshots display a confidence threshold of 0.25 and an IoU threshold of 0.70, together with task controls and runtime readings.

These exhibits document the prototype's visible behavior. The available material does not identify the device model, annotated test-set size, model input resolution, execution backend, or timing sample duration. Consequently, the reported values are transcriptions of displayed readings, and the visual demonstrations are not presented as a controlled accuracy benchmark.

Observed Configuration and Evaluation Boundary

The visible prototype configuration provides a useful functional baseline for interpreting the remaining results. The camera preview, selected task mode, confidence threshold, IoU threshold, overlay type, and displayed runtime fields form the minimum context needed to understand each demonstration. Recording these values with every future run will make comparisons between scenes more reliable and will distinguish a change in model behavior from a change in interface settings.

The current evidence also separates what can be observed directly from what still requires testing. An overlay confirms that a frame passed through capture, inference, coordinate mapping, and display; it does not confirm class accuracy, stable tracking, or suitability for operational deployment. A proper evaluation should therefore pair the screenshots with the device configuration, model file, input resolution, dataset split, repeated timing samples, and clearly defined success criteria.

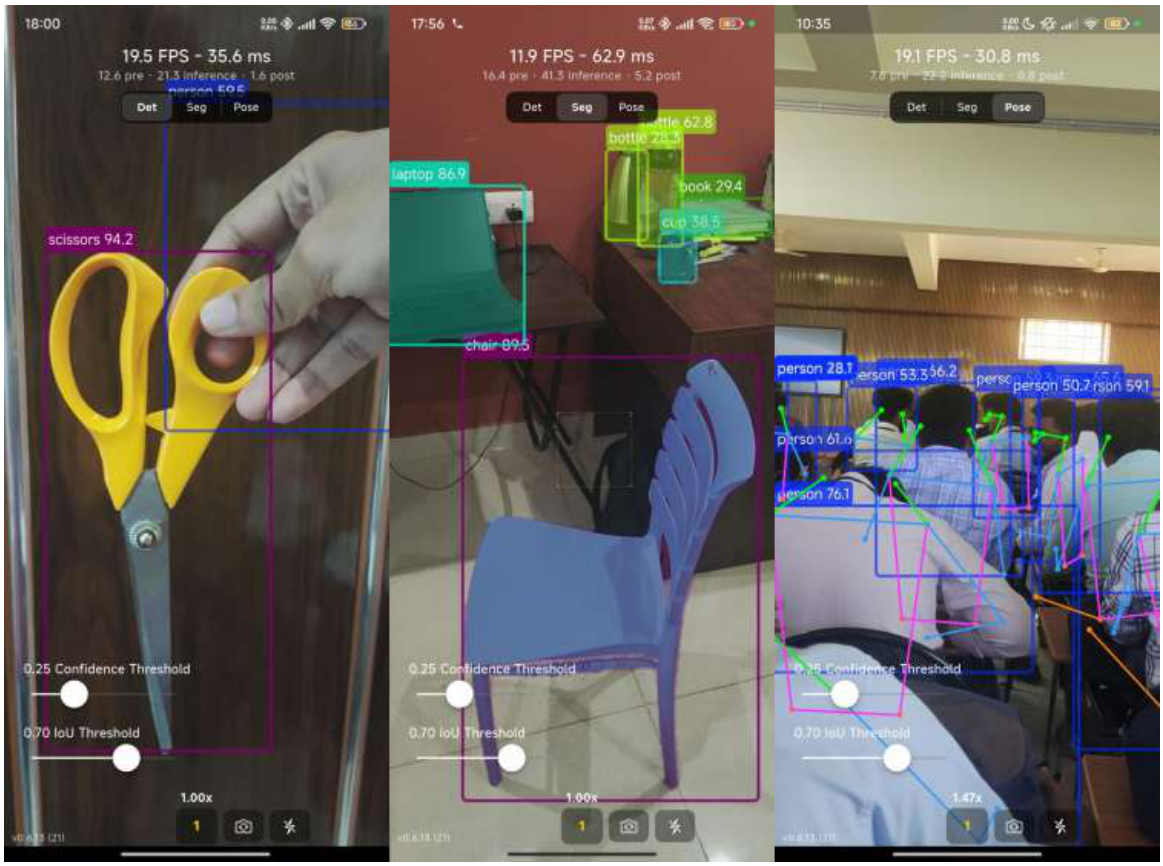
The demonstrations also provide a useful separation between interface evidence and model evidence. The presence of a live preview and an overlay confirms that the camera, inference call, coordinate mapping, and rendering path were connected for that frame. The class name and confidence value show what the deployed model displayed, but they do not prove that the class was correct. Similarly, a smooth-looking preview suggests that the application remained responsive during the capture, but it does not establish a sustained frame rate over a long run.

This distinction is important for interpreting the chapter responsibly. The result is a record of what the prototype visibly produced under the available demonstration conditions. It is not a claim that the system has already been trained on a combat dataset, validated against field imagery, or qualified for operational deployment. The evidence is still useful because it identifies the working capture-to-display path and makes the next evaluation steps concrete.

7.2 Visual Output

Detection, Segmentation, and Pose Modes

Three portrait demonstrations are shown in Snap 7.1. The detection exhibit labels scissors with a displayed score of 94.2. The segmentation exhibit shows colored regions and labels for ordinary indoor objects. The pose exhibit combines person boxes with connected body keypoints in a group scene. Together, these outputs demonstrate that the application can present distinct computer-vision tasks through the same camera interface.



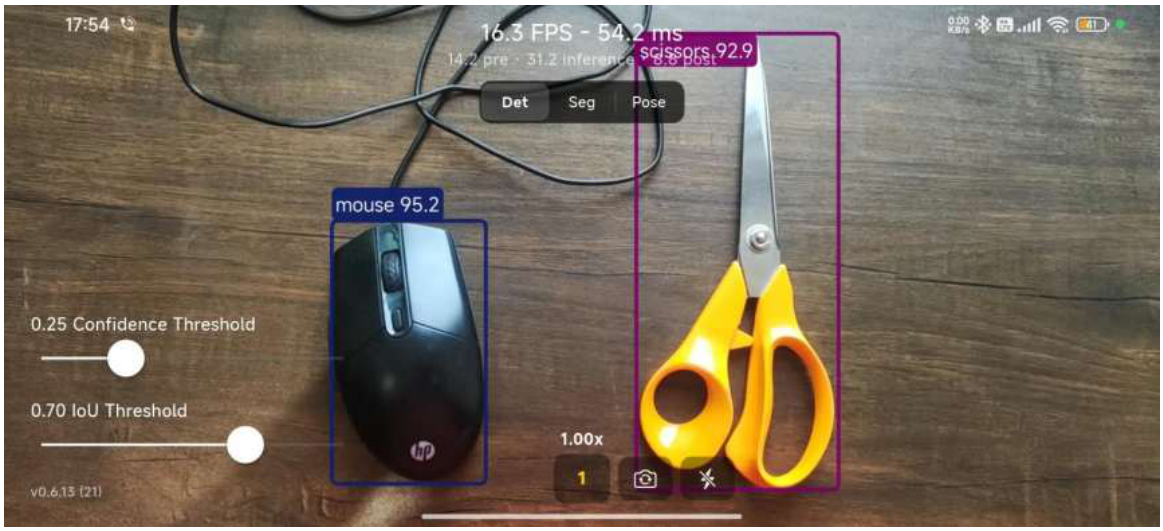
Snap 7.1: Detection, segmentation, and pose outputs from the demonstrated prototype.

Landscape Object Detection

The tabletop exhibit in Snap 7.2 displays a mouse and scissors with scores of 95.2 and 92.9 respectively. The boxes correspond to the visible objects in the landscape view. These values are model confidence scores for the displayed predictions; they are not percentages of correct detections over a test dataset.

Segmentation and Group Pose Visualization

The interior scene is shown in Snap 7.3. The screenshot includes region overlays and labels for a couch, vase, laptop, and bottle. The displayed bottle score is 28.6, which is above the



Snap 7.2: Landscape object-detection output from the demonstrated prototype.

configured 25% confidence threshold but substantially lower than the laptop score of 90.3. This illustrates why per-object confidence and visual checking matter when interpreting the output.



Snap 7.3: Indoor segmentation output from the demonstrated prototype.

The group scene in Snap 7.4 contains multiple person boxes and connected pose keypoints, including partially occluded subjects. This establishes that multi-person overlays were displayed in the demonstrated scene. It does not quantify pose accuracy, tracking consistency, or performance under battlefield conditions.

Cross-Orientation and Task Comparison

The visual exhibits cover both portrait and landscape presentation. Portrait scenes make the vertical camera workflow and the stacking of interface controls easy to inspect, while landscape scenes show how the same output path behaves when the available horizontal



Snap 7.4: Multi-person pose output from the demonstrated prototype.

space changes. The comparison is particularly relevant for a wearable or helmet-mounted concept, where the camera orientation may change as the operator moves or changes the viewing posture.

Detection mode provides the clearest first check because the output is represented by boxes, labels, and confidence values. Segmentation mode adds a denser visual layer and therefore tests whether masks remain aligned without obscuring the scene. Pose mode adds keypoints and connections, making it useful for checking multi-person rendering and partial occlusion. Across the exhibits, the prototype shows that these modes can be presented through the same interface, but the screenshots do not rank one mode as more accurate or more suitable for field use.

The displayed confidence values also illustrate why visual interpretation should accompany numerical readings. A high score may make an overlay appear convincing, while a lower score can still be useful when the object is visible and the threshold permits it to be displayed. Conversely, a visually plausible box can still be a false detection. A controlled assessment would compare these displays with ground-truth annotations rather than treating the confidence number as an accuracy percentage.

7.3 Displayed Runtime Measurements

The frame-rate and processing-time readings visible in the six exhibits are recorded in Table 7.1. The values span 11.9–21.3 FPS and 30.8–62.9 ms of displayed processing time. They refer to different scenes and task modes, so they should not be treated as an average over one uniform experiment or used to rank model variants without additional test controls.

Table 7.1: Displayed Runtime Readings

Exhibit	View	FPS	Time (ms)
Exhibit A: Portrait scissors detection	Portrait	19.5	35.6
Exhibit B: Portrait segmentation	Portrait	11.9	62.9
Exhibit C: Portrait group pose	Portrait	19.1	30.8
Exhibit D: Landscape mouse and scissors	Landscape	16.3	54.2
Exhibit E: Landscape interior segmentation	Landscape	16.0	41.3
Exhibit F: Landscape group pose	Landscape	21.3	37.9

The landscape segmentation screenshot decomposes its 41.3 ms reading into 5.1 ms preprocessing, 32.6 ms inference, and 3.6 ms post-processing. The group-pose screenshot similarly shows 10.2 ms, 25.2 ms, and 2.5 ms, totaling 37.9 ms. In both snapshots, inference is the largest displayed component, making model execution an important target for further profiling.

The displayed FPS is not simply the reciprocal of the processing-time field. For instance, 37.9 ms corresponds to approximately 26.4 processing cycles per second, whereas the screenshot reports 21.3 FPS. Capture, scheduling, rendering, and the application's averaging conventions can affect the relationship between these indicators. The timing windows are not defined, so the processing-time field is not claimed to measure end-to-end camera-to-display latency.

The six readings are best interpreted as configuration-specific observations. The portrait segmentation reading of 11.9 FPS and 62.9 ms is not directly comparable to the landscape group-pose reading of 21.3 FPS and 37.9 ms because the task, scene, orientation, and processing path differ. Likewise, the preprocessing, inference, and post-processing components shown in the exhibits identify where time was displayed for those frames, but they do not provide a distribution across repeated samples. A useful follow-up experiment would collect a fixed number of readings for each task and orientation, then report mean, minimum, maximum, and spread using the same device and model.

The current table therefore supports a practical engineering observation: inference is a visible contributor to processing time, while the relationship between displayed FPS and component timings depends on capture, scheduling, rendering, and averaging behavior. It does not support a claim of stable real-time performance under all conditions. That limitation is retained so that future measured results can be added without contradicting the present evidence.

Observation-Based Comparison

The six displayed readings can still be compared descriptively when their context is retained. The portrait segmentation exhibit has the lowest displayed FPS and the largest displayed processing time in the table. This is consistent with segmentation presenting a denser output than a simple detection box, but the available evidence does not isolate the task from the scene, device state, or application averaging. It is therefore an observation about the supplied configuration, not a measured general property of every segmentation run.

The landscape group-pose exhibit has the highest displayed FPS among the six readings, while the portrait group-pose exhibit reports a lower displayed FPS. The difference shows why orientation should be recorded, but it does not prove that landscape processing is intrinsically faster. The scenes may contain different numbers of visible people, the preview dimensions may differ, and the timing field may use a rolling or task-specific calculation. A controlled comparison would hold the device, scene, model, resolution, and number of samples constant while changing only the orientation.

The visual exhibits provide a second form of comparison. Detection outputs are comparatively sparse and easy to inspect, segmentation outputs cover regions of the image, and pose outputs add several keypoints and connecting lines. As visual complexity increases, readability and overlay placement become important interface concerns even when the underlying model call is unchanged. The screenshots support this interface-level comparison, but they do not establish which task is more accurate or more suitable for deployment.

Recommended Evaluation Protocol

The next evaluation should begin with a documented test set that represents the intended operating conditions. It should identify the object classes, image sources, scene conditions, distance and scale ranges, lighting variation, occlusion cases, and the annotation procedure. A fixed test split must be kept separate from any training or fine-tuning data. If the project later introduces combat-specific classes, the class definitions and annotation policy should be stated before scores are calculated.

For accuracy, predictions should be matched against the annotations using a declared IoU rule and the same confidence-processing policy across runs. Precision, recall, and mAP should then be reported with the dataset size and split, rather than as a single unexplained percentage. For pose or segmentation, the appropriate task-specific metrics should be selected instead of reusing an object-detection score. The report should also identify false positives and missed detections because a high-level aggregate can hide failures on small, distant, or partially occluded objects.

For performance, the device model, Android version, build, model export, input resolution, execution delegate, battery condition, and thermal state should be recorded. Each task

and orientation should be run for a warm-up period followed by a fixed measurement interval. The resulting samples should be reported with a mean and spread, together with minimum and maximum values when useful. Camera-to-display latency should be measured separately from model processing time if the system is intended for responsive operator feedback. Power and thermal measurements should use a documented instrument or system method.

The test procedure should include both normal and adverse conditions. Normal cases can verify stable overlays and predictable controls, while adverse cases can cover motion, glare, low light, clutter, occlusion, changing distance, and orientation transitions. The evaluator should record whether the application recovers after permission interruption, a dropped frame, or a temporary model error. These observations are part of reliability evidence and should be discussed alongside numerical metrics.

Until this protocol is completed, the current results should be described as a functional demonstration with configuration-specific runtime observations. That description is useful because it identifies the working capture, inference, mapping, and rendering path while leaving a clear method for the next study. It also prevents the displayed confidence value from being mistaken for accuracy and prevents a single successful frame from being described as sustained real-time operation.

7.4 Discussion and Limitations

The demonstration establishes a functional foundation of live camera input, on-device inference, task-specific visualization, configurable thresholds, and displayed runtime information, supporting continued development of a portable situational-awareness application; however, its classes are predominantly ordinary objects and people, not a validated combat-specific threat vocabulary.

Quantitative accuracy evaluation requires an annotated dataset and defined test split; precision, recall, and mAP are not reported because the available material lacks those measurements. Sustained FPS, latency distributions, power consumption, and thermal behavior likewise require repeated runs on a documented device. Literature-review results belong to those studies, not to VeerDrishti.

The results can consequently be organized into three evidence levels. Directly observed evidence includes camera preview operation, object overlays, segmentation regions, pose keypoints, threshold controls, and displayed runtime fields. Implementation-supported behavior includes the documented capture, rotation, preprocessing, inference, and rendering sequence. Claims that still require new evidence include combat-specific recognition, accuracy against annotated data, sustained performance, power and thermal behavior, tracking consistency, and operational reliability. This classification keeps the discussion useful while preventing a prototype screenshot from being presented as a complete field evaluation.

Further evaluation should cover small or distant objects, occlusion, camera motion, illumination changes, false or missed detections, and overlay alignment. Combat-specific fine-tuning, thermal fusion, open-vocabulary recognition, and persistent event storage remain future work requiring separate implementation, evaluation, and controlled robustness testing.

Chapter 8

CONCLUSION

VeerDrishti has progressed from a proposed combat-vision framework to an Android prototype that connects camera capture, on-device inference, and visual output. The implementation combines CameraX for live image acquisition, YOLO26 Nano task variants for the demonstrated vision modes, Flutter YOLOView for the application-facing integration, and Logcat for development diagnostics. The presented screenshots demonstrate object detection, segmentation, and multi-person pose overlays in portrait and landscape views, with configurable confidence and IoU thresholds and displayed runtime indicators. The implementation also treats camera orientation as part of the processing path: a frame must be rotated, mapped, resized, and rendered consistently so that a box, mask, or keypoint remains attached to the visible object after the device changes orientation. Together, these elements establish a practical capture-to-display foundation rather than an isolated model demonstration. They also provide concrete evidence for the Implementation and Result chapters, where the integration boundaries, visible outputs, runtime readings, and evidence limitations are recorded separately.

The observed runtime readings and visual exhibits do not establish battlefield accuracy, sustained performance, or a reduction in casualties. Those outcomes remain project objectives that require combat-relevant data, a documented device and model configuration, repeatable test cases, and controlled evaluation. The next development steps are therefore to preserve the selected build and execution configuration, collect repeated measurements for each task and orientation, compare predictions with ground-truth annotations, and report accuracy, latency, variability, power, and thermal behavior using defined procedures. The interface should also be refined for readability when several objects, masks, or keypoints appear together, while failure paths such as denied camera permission, missing models, orientation changes, occlusion, and interrupted frames should be exercised deliberately. Thermal fusion, open-vocabulary recognition, persistent event recording, and combat-specific fine-tuning remain possible extensions rather than demonstrated features. By grounding further development in measured evidence and reproducible checks, VeerDrishti can advance toward a more reliable, portable visual decision-support system without overstating what the current prototype has already proved.

REFERENCES

[1] X. Zhuang, D. Li, Y. Wang, and K. Li, "Military target detection method based on EfficientDet and Generative Adversarial Network," *Engineering Applications of Artificial Intelligence*, vol. 132, 107896, Jun. 2024. DOI: 10.1016/j.engappai.2024.107896.

[2] C. Konar, D. Das, D. S., K. Jc, and S. Singh, "Edge-optimized threat detection with Florence v2 for autonomous surveillance in resource-constrained environments," *Peer J Computer Science*, vol. 12, 3579, Mar. 2026. DOI: 10.7717/peerj-cs.3579.

[3] A. A. S. Matta and V. P. C. S. R. Manukonda, "Spatiotemporal deep learning framework for predictive behavioral threat detection in surveillance footage," *Frontiers in Big Data*, vol. 9, Feb. 2026. DOI: 10.3389/fdata.2026.1770989.

[4] N. Li, P. Xin, J. Tian, *et al.*, "PDGV-DETR: Object detection for secure on-site weapon and personnel location based on dynamic convolution and cross-scale semantic fusion," *Sensors*, vol. 26, no. 5, 1542, Feb. 2026. DOI: 10.3390/s26051542.

[5] A. M. Garcia-Rubio, L. Heck, K. M. Schweitzer, R. M. Bateman, A. Alaeddini, and K. K. Castillo-Villar, "Detecting concept drift in object detection models: A collaborative AI-human approach for defense applications," *IEEE Access*, vol. 14, pp. 7412–7429, Jan. 2026. DOI: 10.1109/ACCESS.2026.3652106.

[6] X. Xiang, G. Zhou, B. Niu, *et al.*, "Infrared-visible image fusion meets object detection: Towards unified optimization for multimodal perception," *Remote Sensing*, vol. 17, no. 21, 3637, Nov. 2025. DOI: 10.3390/rs17213637.

[7] Z. Shou and D. Zhu, "Multi-modal action recognition via advanced image fusion techniques for cyber-physical systems," *Frontiers in Physics*, vol. 13, 1576591, Aug. 2025. DOI: 10.3389/fphy.2025.1576591.

[8] Z. Song, Y. Zhang, and A. A. R. M. A. Ebayyeh, "EDNet: Edge-optimized small target detection in UAV imagery — faster context attention, better feature fusion, and hardware acceleration," in *2024 IEEE Smart World Congress (SWC)*, IEEE, Dec. 2024, pp. 829–838. DOI: 10.1109/SWC62898.2024.00141.

[9] S. D. N. and D. M., "Real-time versatile object detection with high accuracy and reduced information loss during wartime," in *2024 International Conference on Integration of Emerging Technologies for the Digital World (ICIETDW)*, IEEE, Sep. 2024, pp. 1–7. DOI: 10.1109/ICIETDW61607.2024.10939768.

[10] V. Kaya, S. Tuncer, and A. Baran, "Detection and classification of different weapon types using deep learning," *Applied Sciences*, vol. 11, no. 16, 7535, Aug. 2021. DOI: 10.3390/app11167535.

[11] Ankita, S. Rani, H. Babbar, S. Coleman, A. Singh, and H. M. Aljahdali, "An efficient and lightweight deep learning model for human activity recognition using smartphones," *Sensors*, vol. 21, no. 11, 3845, Jun. 2021. DOI: 10.3390/s21113845.

ICSTIM 2026 ABSTRACT SUBMISSION

The following page reproduces the abstract submitted for ICSTIM 2026, the International Conference on Sustainable Technologies, Innovation and Management. The paper was suggested for submission under the title *"VeerDrishti: A Computer Vision Framework for Real-Time Situational Awareness and Visual Threat Assessment."*

ICSTIM114

VeerDrishti: A Computer Vision Framework for Real-Time Situational Awareness and Visual Threat Assessment

*Gaurav Chauhan, Gaurav Singh, Sapna N
Department of Computer Science and Engineering, Bangalore Technological Institute
Corresponding Author Email: *gaurav1bh23cs024@btibangalore.org

Abstract

In high-stress combat environments, operators must process a lot of visual information while dealing with constant positioning, movement, poor visibility, stress and mental fatigue. Continuous manual monitoring makes it difficult to spot threats, targets and points of interest quickly. It can slow decision-making leading to operational difficulties and casualties. VeerDrishti is an AI-powered computer vision framework that aims to provide real-time situational awareness and automated visual threat assessment. The system captures live video using a camera module and processes each frame using a on-device detection model. The detection pipeline does frame pre-processing, real-time inference and post-processing before it shows the detected objects with bounding boxes, labels and confidence scores on an augmented reality interface. The framework focuses on low-latency processing so that visual detection happens directly on edge hardware without an external computing infrastructure. So far tests have shown real-time operation at 18-25 frames per second with low latency on inexpensive low-end hardware under different conditions. Further evaluation on field conditions has to be done to assess accuracy, robustness and reliability in real-world operational environments.

Keywords: *Artificial intelligence, computer vision, real-time inference, object detection, classification, low-latency, edge deployment, augmented reality*

DRDO SYNOPSIS SUBMISSION

The following page reproduces the synopsis mailed to the DRDO Students Development Program executive. The response is awaited.

VeerDrishti

VeerDrishti is an AI-powered combat vision system engineered to deliver real-time situational awareness and automated visual threat assessments for ground forces in high-stress tactical operations.

Project Overview

- **Primary Objective**: Automate visual monitoring to reduce cognitive fatigue, prevent fatal human errors, and minimize casualties among military personnel.
- **Deployment Hardware**: Designed for edge deployment on low-cost wearable hardware, such as helmet-mounted smartphones or Raspberry Pi units, allowing reliable performance in signal-denied zones.

Technical Methodology

- **Lightweight Detection Models**: Employs edge-optimized deep-learning architectures like YOLO to ensure low-latency, real-time object classification.
- **Multimodal Spectrum Fusion**: Can merge visible RGB texture data with thermal infrared (IR) edge signals through region-attention mechanisms to maintain detection accuracy in low-light, night, or obscured settings.
- **Generative Data Augmentation**: Utilizes Generative Adversarial Networks (GANs) to synthesize military-specific datasets, addressing data scarcity in tactical vision models.
- **Zero-Shot Threat Identification**: Leverages vision-language models to identify novel or unseen threats using natural language supervision.
- **Augmented Reality (AR) Interface**: Applies an instant AR visual overlay to highlight enemy combatants, weapons, and vehicles on-screen while logging local mission metadata for post-operation auditing.

Feasibility & Global Alignment

- **Triple-Dimension Feasibility**: Assesses technical feasibility via mobile GPU performance, operational feasibility through low cognitive overhead, and economic feasibility by leveraging commercial mobile hardware and open-source frameworks.
- **UN SDG Alignment**: Supports UN Sustainable Development Goal 16 (Peace, Justice, and Strong Institutions) by mitigating operational risk (Target 16.1), retaining mission video logs for legal and operational reviews (Target 16.3), and strengthening national defense capabilities (Target 16.A).

APPENDIX A

SOURCE CODE LISTING (1/3)

```
// Ultralytics YOLO Android wrapper
// AGPL-3.0 License - https://ultralytics.com/license
package com.ultralytics.yolo

import android.content.Context
import android.graphics.*
import android.net.Uri
import android.provider.MediaStore
import android.util.Log
import androidx.camera.core.ImageProxy
import kotlinx.coroutines.*
import java.io.IOException
import java.net.URL
class YOLO(
    private val context: Context,
    private val modelPath: String,
    val task: YOLOTask,
    private val labels: List<String> = emptyList(),
    private val useGpu: Boolean = true,
    private var numItemsThreshold: Int = 30,
    private val classifierOptions: Map<String, Any>? = null
) {
    private val tag = "YOLO"
    private var closed = false
    private val predictorDelegate = lazy(LazyThreadSafetyMode.NONE) {
        Predictor.create(context, modelPath, task, labels, useGpu,
            numItemsThreshold, classifierOptions) as BasePredictor
    }
    private val predictor: BasePredictor
        get() { check(!closed); return predictorDelegate.value }
    @Synchronized
    fun predict(bitmap: Bitmap, rotateForCamera: Boolean = false): YOLOResult {
        predictor.includeRawMaskData = true
        val result = predictor.predict(bitmap, bitmap.width, bitmap.height,
            rotateForCamera, isLandscape = false)
        return result.copy(originalImage = bitmap,
            annotatedImage = drawAnnotations(bitmap, result, rotateForCamera),
            accelerator = predictor.accelerator)
    }
    @Synchronized
    fun predict(imageProxy: ImageProxy): YOLOResult? {
        val bitmap = ImageUtils.toBitmap(imageProxy) ?: return null
        val rotation = imageProxy.imageInfo.rotationDegrees
        predictor.cameraRotationDegrees = rotation
        val result = predictor.predict(bitmap, imageProxy.width, imageProxy.height, true, false)
        return result.copy(originalImage = bitmap,
            annotatedImage = drawAnnotations(bitmap, result, true, rotation),
            accelerator = predictor.accelerator)
    }
    @Synchronized
    fun predict(imageUri: Uri): YOLOResult? = try {
        val bitmap = MediaStore.Images.Media.getBitmap(context.contentResolver, imageUri)
        predict(bitmap)
    } catch (error: IOException) {
        Log.e(tag, "Failed to load Uri: ${error.message}"); null
    }
    suspend fun predict(imageUrl: String): YOLOResult? = withContext(Dispatchers.IO) {
        try {
            val bitmap = BitmapFactory.decodeStream(URL(imageUrl).openStream())
            predict(bitmap)
        } catch (error: IOException) {
            Log.e(tag, "Failed to load URL: ${error.message}"); null
        }
    }
```

SOURCE CODE LISTING (2/3)

```
    private fun calculateSmartLabelRect(
        box: RectF, labelWidth: Float, labelHeight: Float,
        viewWidth: Float, viewHeight: Float
    ): RectF {
        var left = box.left; var top = box.top - labelHeight
        var right = left + labelWidth; var bottom = box.top
        if (top < 0) { top = box.top; bottom = top + labelHeight }
        if (left < 0) { left = 0f; right = labelWidth }
        if (right > viewWidth) { right = viewWidth; left = right - labelWidth }
        if (bottom > viewHeight) { bottom = viewHeight; top = bottom - labelHeight }
        return RectF(left, top, right, bottom)
    }

    private fun drawAnnotations(bitmap: Bitmap, result: YOLOResult,
        rotateForCamera: Boolean = true, rotationDegrees: Int? = null): Bitmap {
        val degrees = rotationDegrees ?: 90
        val rotated = degrees % 180 != 0
        val output = if (rotateForCamera) Bitmap.createBitmap(
            if (rotated) bitmap.height else bitmap.width,
            if (rotated) bitmap.width else bitmap.height, Bitmap.Config.ARGB_8888
        ) else bitmap.copy(Bitmap.Config.ARGB_8888, true)
        val canvas = Canvas(output)
        if (rotateForCamera) { canvas.save()
            canvas.translate(output.width / 2f, output.height / 2f)
            canvas.rotate(degrees.toFloat())
            canvas.drawBitmap(bitmap, -bitmap.width / 2f, -bitmap.height / 2f, null)
            canvas.restore() }
        val paint = Paint().apply { style = Paint.Style.STROKE }
        val sx = output.width / result.origShape.width.toFloat()
        val sy = output.height / result.origShape.height.toFloat()
        fun scale(box: RectF) = RectF(box.left * sx, box.top * sy,
            box.right * sx, box.bottom * sy)

        when (task) {
            YOLOTask.DETECT, YOLOTask.SEGMENT, YOLOTask.POSE -> {
                result.boxes.forEach { box ->
                    paint.color = ultralyticsColors[box.index % ultralyticsColors.size]
                    val bounds = scale(box.xywh)
                    canvas.drawRoundRect(bounds, 12f, 12f, paint)
                    val text = "${box.cls} ${(box.conf * 100).toInt()}%"
                    val label = calculateSmartLabelRect(bounds,
                        paint.measureText(text) + 16f, 48f,
                        output.width.toFloat(), output.height.toFloat())
                    paint.style = Paint.Style.FILL
                    canvas.drawRoundRect(label, 12f, 12f, paint)
                    paint.color = Color.WHITE; canvas.drawText(text,
                        label.left + 8f, label.bottom - 8f, paint)
                    paint.style = Paint.Style.STROKE
                }
                // Full implementation also draws masks, keypoints, and skeletons.
            }
            YOLOTask.SEMANTIC, YOLOTask.DEPTH -> {
                result.semanticMask?.let { drawMask(canvas, it, output) }
                result.depthMap?.let { drawDepth(canvas, it, output) }
            }
            YOLOTask.CLASSIFY -> drawTopFive(canvas, result, paint)
            YOLOTask.OBB -> drawOrientedBoxes(canvas, result, paint, sx, sy)
        }
        return output
    }

    @Synchronized fun setConfidenceThreshold(value: Double) =
        predictor.setConfidenceThreshold(value)
    @Synchronized fun setIouThreshold(value: Double) = predictor.setIouThreshold(value)
    @Synchronized fun setNumItemsThreshold(max: Int) {
        numItemsThreshold = max; predictor.setNumItemsThreshold(max)
    }
    fun getConfidenceThreshold(): Float = predictor.getConfidenceThreshold().toFloat()
    fun getIouThreshold(): Float = predictor.getIouThreshold().toFloat()
    @Synchronized fun close() {
        closed = true
        if (predictorDelegate.isInitialized()) predictorDelegate.value.close()
    }
}
```

SOURCE CODE LISTING (3/3)

```
private fun drawMask(canvas: Canvas, mask: Bitmap, output: Bitmap) {
    val paint = Paint(Paint.ANTI_ALIAS_FLAG).apply {
        alpha = 96
        xfermode = PorterDuffXfermode(PorterDuff.Mode.SRC_OVER)
    }
    val scaled = Bitmap.createScaledBitmap(mask, output.width,
        output.height, true)
    canvas.drawBitmap(scaled, 0f, 0f, paint)
}

private fun drawDepth(canvas: Canvas, depth: Bitmap, output: Bitmap) {
    val paint = Paint(Paint.ANTI_ALIAS_FLAG)
    val scaled = Bitmap.createScaledBitmap(depth, output.width,
        output.height, true)
    paint.colorFilter = ColorMatrixColorFilter(ColorMatrix(floatArrayOf(
        1f, 0f, 0f, 0f, 0f, 1f, 0f, 0f, 0f,
        0f, 0f, 1f, 0f, 0f, 0f, 0f, 1f, 0f)))
    canvas.drawBitmap(scaled, 0f, 0f, paint)
}

private fun drawTopFive(canvas: Canvas, result: YOLOResult, paint: Paint) {
    result.topClassifications.take(5).forEachIndexed { index, item ->
        val y = 48f + index * 42f
        paint.style = Paint.Style.FILL
        paint.color = Color.WHITE
        paint.textSize = 28f
        canvas.drawText("${item.label}: ${item.confidence}", 24f, y, paint)
    }
}

private fun drawOrientedBoxes(canvas: Canvas, result: YOLOResult,
    paint: Paint, sx: Float, sy: Float) {
    result.boxes.forEach { box ->
        paint.style = Paint.Style.STROKE
        paint.strokeWidth = 4f
        paint.color = ultralyticsColors[box.index % ultralyticsColors.size]
        val points = box.corners.map { PointF(it.x * sx, it.y * sy) }
        if (points.size >= 4) {
            val path = Path().apply {
                moveTo(points[0].x, points[0].y)
                points.drop(1).forEach { lineTo(it.x, it.y) }
                close()
            }
            canvas.drawPath(path, paint)
        }
    }
}

data class DetectionSettings(
    val confidence: Float = 0.25f,
    val iou: Float = 0.45f,
    val maximumItems: Int = 30,
    val useGpu: Boolean = true
)

fun applySettings(settings: DetectionSettings) {
    setConfidenceThreshold(settings.confidence.toDouble())
    setIouThreshold(settings.iou.toDouble())
    setNumItemsThreshold(settings.maximumItems)
}
```

APPENDIX B

PROJECT PHOTOGRAPHS



Project group with the project guide



Project photograph 1



Project photograph 2